PYTHON FOR DATA SCIENCE · LESSON 27 OF 29

DSA II: Stacks, Queues, Linked Lists, Recursion, Sorting and Binary Search

Learn linear data structures (stack, queue, deque, linked list), recursion and backtracking, the main sorting algorithms with their trade-offs, and binary search including "search on the answer", with interview problems such as valid parentheses, next greater element, reverse a linked list and search in a rotated array.

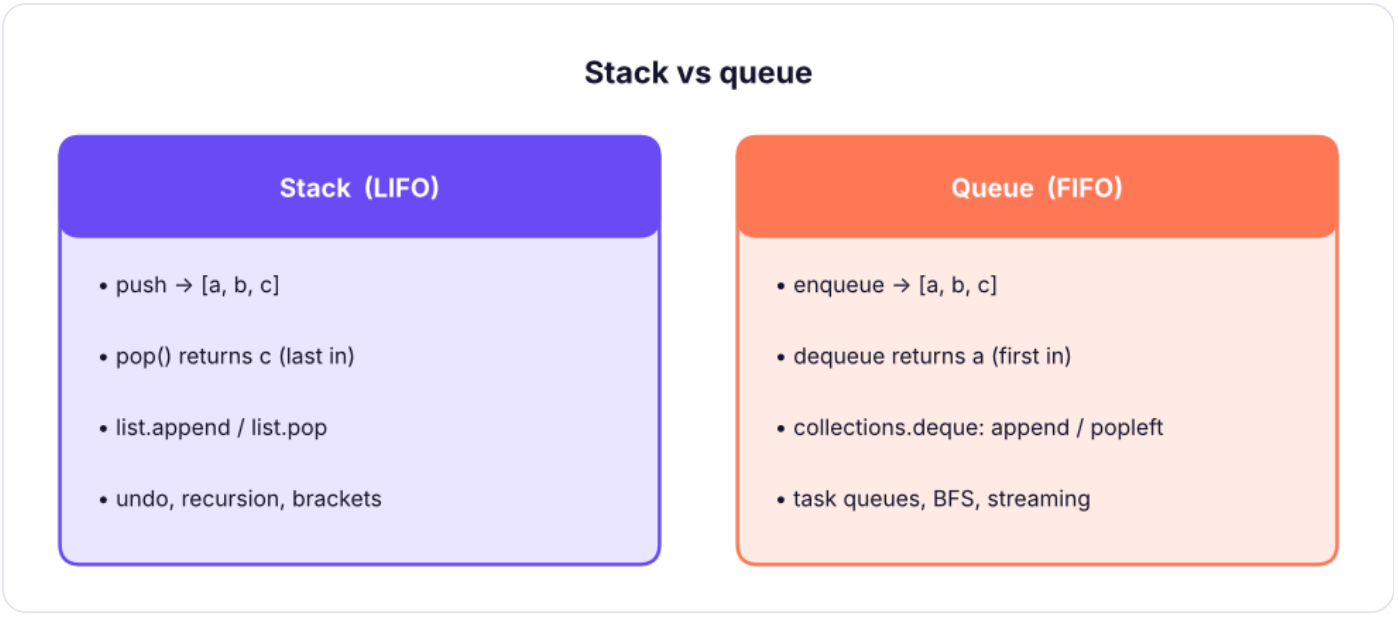
DSA for data roles (added) · Python 3.13 · Intermediate · 120 min

By the end of this lesson you can

- Implement and use stacks, queues and deques, and solve bracket and monotonic-stack problems
- Build and manipulate a singly linked list (reverse, detect a cycle, find the middle)
- Write recursive solutions with base cases and use backtracking for combinations
- Explain bubble, insertion, merge and quick sort and their complexities
- Apply binary search to sorted arrays, boundaries and answer spaces

Stacks: last in, first out

A **stack** adds and removes at the same end (LIFO). In Python a list works: `append` to push, `pop()` to pop, both O(1). Uses: undo, browser back, function calls, parsing brackets.



Stack (LIFO) vs queue (FIFO).

Python

```
def valid_parentheses(s):
    pairs, stack = {")": "(", "]": "[", "}": "{"}, []
    for ch in s:
        if ch in "([{": stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]: return False
    return not stack
print([valid_parentheses(x) for x in ["()[]{}", "([)]", "{[()]}", "((", "SUM(IF(a>1,
(b),c))"]])

def eval_rpn(tokens):                          # reverse Polish notation: "3 4 + 2 *" = 14
    st = []
    for t in tokens:
        if t in "+-*/":
            b, a = st.pop(), st.pop()
            st.append({"+": a + b, "-": a - b, "*": a * b, "/": int(a / b)}[t])
        else: st.append(int(t))
    return st[0]
print(eval_rpn("3 4 + 2 *".split()))
```

OUTPUT

```
[True, False, True, False, True]
14
```

Monotonic stack

Keep the stack sorted to answer "next greater/smaller element" in O(n) instead of O(n²).

Python

```
def next_greater(nums):
    res, st = [-1] * len(nums), []           # st holds indices with decreasing values
    for i, x in enumerate(nums):
        while st and nums[st[-1]] < x: res[st.pop()] = x
        st.append(i)
    return res
def days_until_warmer(temps):
    res, st = [0] * len(temps), []
    for i, t in enumerate(temps):
        while st and temps[st[-1]] < t:
            j = st.pop(); res[j] = i - j
        st.append(i)
    return res
print(next_greater([2, 1, 2, 4, 3]), days_until_warmer([73, 74, 75, 71, 69, 72, 76, 73]))
```

OUTPUT

```
[4, 2, 4, -1, -1] [1, 1, 4, 2, 1, 1, 0, 0]
```

Queues and deques

Python

```
from collections import deque
q = deque(); q.append("job1"); q.append("job2"); q.appendleft("urgent")
print(q.popleft(), q.popleft(), list(q))
recent = deque(maxlen=3)                     # keeps only the last 3 items: a rolling buffer
for x in range(1, 7): recent.append(x)
print(list(recent))
def max_sliding_window(nums, k):             # monotonic deque, O(n)
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] ⩽ x: dq.pop()
        dq.append(i)
        if dq[0] ⩽ i - k: dq.popleft()
        if i ⩾ k - 1: out.append(nums[dq[0]])
    return out
print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

OUTPUT

```
urgent job1 ['job2']
[4, 5, 6]
[3, 3, 5, 5, 6, 7]
```

Never use `list.pop(0)` for a queue: it is O(n). Use `deque.popleft()` (O(1)). `queue.Queue` adds thread safety; `heapq` gives a priority queue (next lesson).

Linked lists

A **linked list** stores nodes that point to the next node. Insert/delete at a known node is O(1); access by index is O(n). Python rarely needs them, but interviews love them.

Python

```
class Node:
    def __init__(self, val, nxt=None): self.val, self.next = val, nxt
def build(vals):
    head = None
    for v in reversed(vals): head = Node(v, head)
    return head
def to_list(head):
    out = []
    while head: out.append(head.val); head = head.next
    return out
def reverse(head):
    prev = None
    while head: head.next, prev, head = prev, head, head.next
    return prev
def middle(head):                          # slow/fast pointers
    slow = fast = head
    while fast and fast.next: slow, fast = slow.next, fast.next.next
    return slow.val
def has_cycle(head):                       # Floyd's tortoise and hare
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast: return True
    return False
def merge_two(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.val ⩽ b.val: tail.next, a = a, a.next
        else: tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b; return dummy.next
lst = build([1, 2, 3, 4, 5])
print(to_list(reverse(build([1, 2, 3, 4, 5]))), middle(lst), has_cycle(lst),
to_list(merge_two(build([1, 4, 7]), build([2, 3, 9]))))
cyc = build([1, 2, 3]); cyc.next.next.next = cyc.next; print(has_cycle(cyc))
```

OUTPUT

```
[5, 4, 3, 2, 1] 3 False [1, 2, 3, 4, 7, 9]
True
```

Recursion and backtracking

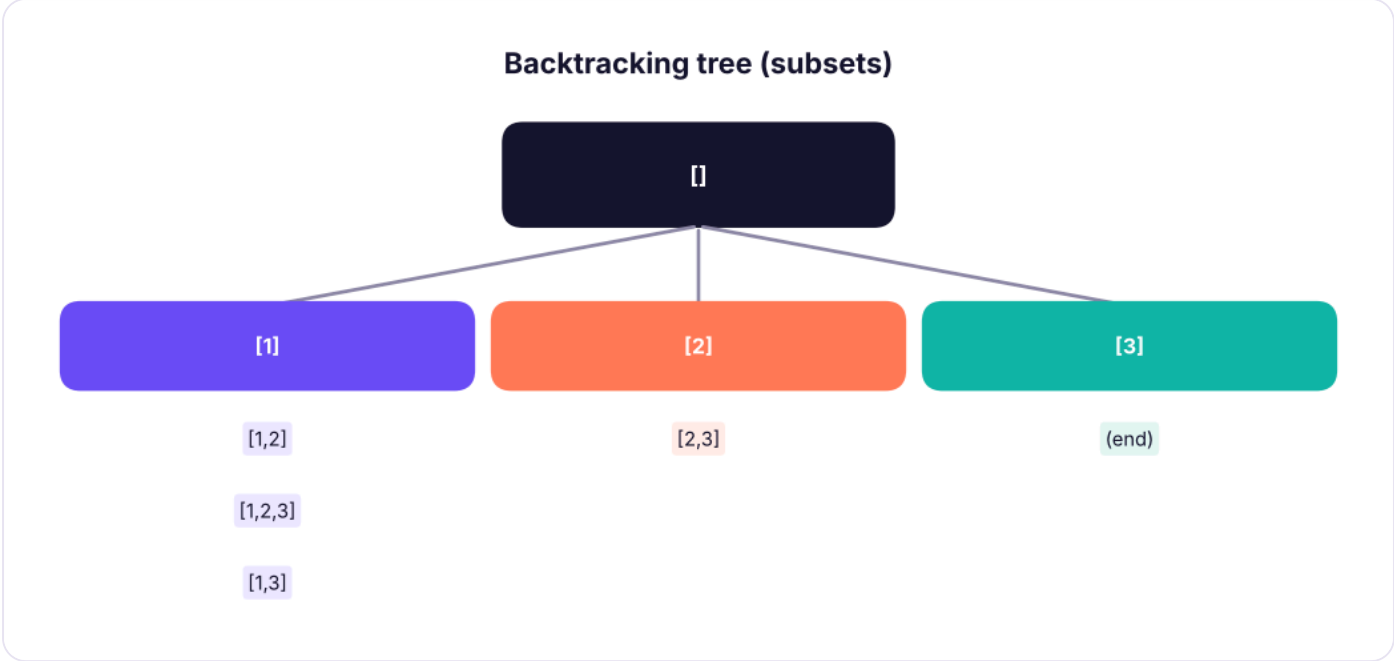
A recursive function solves a problem by calling itself on a smaller version. It needs a **base case** and must move toward it.

Python

```python
def power(x, n):                            # fast exponentiation, O(log n)
    if n == 0: return 1
    half = power(x, n // 2)
    return half * half * (x if n % 2 else 1)
def flatten(nested):
    out = []
    for item in nested: out.extend(flatten(item) if isinstance(item, list) else [item])
    return out
def subsets(nums):                          # backtracking: explore, choose, un-choose
    res = []
    def bt(start, path):
        res.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i]); bt(i + 1, path); path.pop()
    bt(0, []); return res
def permutations(s):
    if len(s) <= 1: return [s]
    return [ch + p for i, ch in enumerate(s) for p in permutations(s[:i] + s[i + 1:])]
print(power(2, 10), flatten([1, [2, [3, 4]], 5]), subsets([1, 2, 3]), permutations("abc"))
```

OUTPUT

```
1024 [1, 2, 3, 4, 5] [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]] ['abc',
'acb', 'bac', 'bca', 'cab', 'cba']
```



The recursion tree for subsets of [1, 2, 3]: each level decides whether to include the next item; $2^3 = 8$ leaves.

Sorting algorithms

Python

```
def bubble_sort(a):
    a = a[:]
    for i in range(len(a)):
        swapped = False
        for j in range(len(a) - 1 - i):
            if a[j] > a[j + 1]: a[j], a[j + 1] = a[j + 1], a[j]; swapped = True
        if not swapped: break
    return a
def insertion_sort(a):
    a = a[:]
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j >= 0 and a[j] > key: a[j + 1] = a[j]; j -= 1
        a[j + 1] = key
    return a
def merge_sort(a):
    if len(a) <= 1: return a
    mid = len(a) // 2; left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: out.append(left[i]); i += 1
        else: out.append(right[j]); j += 1
    return out + left[i:] + right[j:]
def quick_sort(a):
    if len(a) <= 1: return a
    pivot = a[len(a) // 2]
    return quick_sort([x for x in a if x < pivot]) + [x for x in a if x == pivot] +
quick_sort([x for x in a if x > pivot])
data = [38, 27, 43, 3, 9, 82, 10]
print(bubble_sort(data), insertion_sort(data), merge_sort(data), quick_sort(data),
sorted(data) == merge_sort(data))
import random, time
big = [random.random() for _ in range(3000)]
for f in (insertion_sort, merge_sort, quick_sort, sorted):
    t0 = time.perf_counter(); f(big); print(f"{f.__name__:15s} {1000 * (time.perf_counter()
- t0):7.1f} ms")
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82] [3, 9, 10, 27, 38, 43, 82] [3, 9, 10, 27, 38, 43, 82] [3,
9, 10, 27, 38, 43, 82] True
insertion_sort    133.5 ms
merge_sort          3.5 ms
quick_sort          3.5 ms
sorted              0.2 ms
```

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Teaching only
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Great for small / nearly sorted
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Predictable; external sorting
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Fast in practice
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	
Timsort (Python `sorted`)	O(n)	O(n log n)	O(n log n)	O(n)	Yes	Merge + insertion hybrid

Stable means equal keys keep their original order, so you can sort by city, then stably by region. In practice, use `sorted(items, key=...)`.

Binary search

On **sorted** data, halve the search space each step: O(log n). One million items need at most 20 comparisons.

Python

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
import bisect
a = [3, 8, 15, 15, 15, 23, 42]
print(binary_search(a, 23), binary_search(a, 5), bisect.bisect_left(a, 15),
bisect.bisect_right(a, 15), bisect.bisect_right(a, 15) - bisect.bisect_left(a, 15))
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] ⩽ nums[mid]:                    # left half sorted
            if nums[lo] ⩽ target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:                                       # right half sorted
            if nums[mid] < target ⩽ nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0), search_rotated([4, 5, 6, 7, 0, 1, 2], 3))
def min_capacity_to_ship(weights, days):           # "binary search on the answer"
    def ok(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap: d += 1; load = 0
            load += w
        return d ⩽ days
    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo
print(min_capacity_to_ship([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))
grades = ["F", "D", "C", "B", "A"]; cutoffs = [40, 50, 65, 80]
print([grades[bisect.bisect_right(cutoffs, s)] for s in (35, 50, 79, 92)])        # banding
with bisect
```

OUTPUT

```
5 -1 2 5 3
4 -1
15
['F', 'C', 'B', 'A']
```

`bisect` is how pandas `cut` and SQL range lookups work under the hood: find the band for a value in O(log n).

Summary

KEY TAKEAWAYS

- Stack = LIFO (list append/pop); queue = FIFO (`deque` append/popleft); monotonic stacks/deques solve next-greater and window-max in O(n).
- Linked lists: pointer manipulation; slow/fast pointers find the middle and detect cycles.
- Recursion needs a base case; backtracking = choose, explore, un-choose.
- Merge sort is O(n log n) and stable; quick sort is fast on average; insertion sort suits small inputs; use `sorted` .
- Binary search halves the space: O(log n). Use it on sorted arrays, boundaries (`bisect`) and "minimum value that works" answers.

Quick quiz

6 questions · tap an answer to check it

1. Which structure checks balanced brackets?

A Stack

B Queue

C Set

D Heap

Answer: A. Push openings, pop on closings.

2. Why use `deque.popleft()` instead of `list.pop(0)` ?

A It sorts the data

B It is O(1); list.pop(0) is O(n)

C It is thread safe

D No difference

Answer: B. Lists shift every element when removing from the front.

3. How do you find the middle of a linked list in one pass?

A Count then walk

B Slow and fast pointers

C Sort it

D Binary search

Answer: B. When fast reaches the end, slow is in the middle.

4. What is the worst-case complexity of quick sort?

A O(n log n)

B $O(n^2)$

C O(n)

D O(log n)

Answer: B. With bad pivots; randomised pivots make it unlikely.

5. How many comparisons does binary search need for about 1,000,000 sorted items?

A 1,000,000

B 1,000

C About 20

D 2

Answer: C. $\log_2(1{,}000{,}000) \approx 20$.

6. What does a "stable" sort guarantee?

A It never fails

B Items with equal keys keep their original relative order

C O(n log n) always

D It uses no memory

Answer: B. Merge sort and Timsort are stable.

EXERCISE

Practice set

(1) Implement a queue using two stacks. (2) Remove adjacent duplicate letters repeatedly ("abbaca" → "ca"). (3) Find the first and last position of a target in a sorted array with `bisect`. (4) Return all combinations of k numbers from 1..n. (5) Compute the integer square root with binary search.

Show solution (worked answer)

Python

```
class QueueTwoStacks:
    def __init__(self): self.inbox, self.outbox = [], []
    def push(self, x): self.inbox.append(x)
    def pop(self):
        if not self.outbox:
            while self.inbox: self.outbox.append(self.inbox.pop())
        return self.outbox.pop()
def remove_adjacent(s):
    st = []
    for ch in s:
        if st and st[-1] == ch: st.pop()
        else: st.append(ch)
    return "".join(st)
def first_last(a, t):
    i, j = bisect.bisect_left(a, t), bisect.bisect_right(a, t) - 1
    return [i, j] if i <= j else [-1, -1]
def combine(n, k):
    res = []
    def bt(start, path):
        if len(path) == k: res.append(path[:]); return
        for i in range(start, n + 1): path.append(i); bt(i + 1, path); path.pop()
    bt(1, []); return res
def isqrt(x):
    lo, hi = 0, x
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid * mid <= x: lo = mid
        else: hi = mid - 1
    return lo
print(remove_adjacent("abbaca"), first_last([5, 7, 7, 8, 8, 10], 8), combine(4, 2),
isqrt(80))
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain merge sort. Split the list in half recursively until single items remain, then merge sorted halves by repeatedly taking the smaller front item; O(n log n) time, O(n) extra space, stable.

Q: When is binary search applicable? When the data is sorted, or more generally when a yes/no condition is monotonic over the search space (search on the answer).

Q: Stack vs queue in real systems? Stacks: call stacks, undo, expression parsing, DFS. Queues: job and message queues (Kafka, SQS), BFS, rate-limited processing.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.